

Linux and Your Machine's I/O Architecture

A 75 minute walkthrough for Tuesday

Every diagram in Chapters 3 and 8 describes a machine that is, in fact, sitting in front of you right now. Interrupts, the bus hierarchy, and the choice between programmed I/O and DMA are not historical artifacts. They are things a running Linux system reports about itself in real time. Today we look at three of them directly: interrupts, the physical bus tree, and the CPU cost difference between programmed I/O and DMA.

Two screens, one system. Run every command on your own terminal first. Then look at the instructor's screen for the same command. The exact numbers will differ, sometimes a lot, but the shape of what you are looking at, an interrupt table, a bus tree, a CPU that stays idle during a large transfer, will not. That is the point. The concept is the same across an Arch machine, a Gentoo machine, a Void machine, and whatever the instructor is running.

If a command is missing. If `lspci`, `lsusb`, `ethtool`, `iw`, or `hdparm` prints "command not found," the setup checklist sent before class was not completed. See the appendix at the end of this handout for the install command for your distribution, or follow along on the instructor's screen for that step and catch up afterward.

1. Interrupts, live

The book's interrupt cycle, Figures 3.9 through 3.14, describes a processor that checks for a pending interrupt, saves its place, and jumps to a handler. Linux keeps a running count of exactly this, per device and per CPU core.

Open two terminal windows side by side.

In the first, start a live view of the interrupt table.

```
watch -n 1 cat /proc/interrupts
```

If watch is not installed. Use this loop instead, it needs nothing beyond a shell.

```
while true; do clear; cat /proc/interrupts; sleep 1; done
```

In the second window, generate some I/O. Type quickly for a few seconds, move the mouse, or run `ping -c 20 127.0.0.1`. Watch the counts in the first window climb, specifically the rows for your keyboard controller, your input device, or your network interface.

Discussion. Each row is one interrupt source. Each column after the first is a CPU core. The number is how many times that core has serviced that interrupt since boot. This is Figure 3.9 with

numbers attached. Compare your keyboard row with the instructor's. The IRQ numbers will not match, since the exact hardware differs, but the behavior, a count that only rises when that device does work, is identical.

2. The bus hierarchy, live

The textbook's bus diagrams and the PCIe transaction layer discussion in Chapter 3 describe a tree, a root complex at the center with bridges and endpoints hanging off it. That tree is not a simplification for the page. It is what `lspci` reports.

```
lspci -tv
```

Discussion. The top of the tree is the root complex. Indented lines below it are bridges and endpoint devices, your graphics controller, your network card, your storage controller. This is the same topology as the point to point interconnect and PCIe material in Chapter 3.

Now do the same for USB, one of the external interface types Chapter 8 covers.

```
lsusb -tv
```

Discussion. Notice the hub and device structure and the speed listed next to each device. Compare trees across the room. A desktop, a laptop, and the instructor's machine will all show different devices, but the same branching hub structure the chapter describes.

3. Programmed I/O versus DMA, live

Chapter 8's central argument for DMA, Table 8.1 and Figures 8.12 and 8.13, is efficiency. Programmed I/O keeps the processor busy waiting on a device. DMA lets a transfer happen while the processor does something else. We can watch that difference in CPU behavior directly.

First, identify your main storage device.

```
lsblk
```

In one terminal, start a live view of CPU activity.

```
vmstat 1
```

The column to watch is `id`, idle CPU percentage, and `wa`, time spent waiting on I/O.

In a second terminal, create a large file and copy it.

```
dd if=/dev/zero of=~/.bigfile bs=1M count=2048  
cp ~/.bigfile ~/.bigfile-copy
```

Discussion. While the copy runs, the idle column in `vmstat` stays high. The processor is not the one moving the bytes. This is DMA at work, the drawback of programmed I/O that Figure 8.12 is solving, the processor delegates the transfer instead of polling the device in a loop.

Optional, SATA drives with hdparm installed. Run `lsblk` to find your disk device, for example `/dev/sda`. Then `sudo hdparm -I /dev/sda` and look near the top of the output for a line mentioning DMA support. This does not apply to NVMe drives, which are PCIe attached and DMA capable by design. If `lsblk` reports an `nvme0n1` device instead, you already saw it, it is the storage controller you found in the PCIe tree in the previous section.

Clean up when finished.

```
rm ~/bigfile ~/bigfile-copy
```

4. Suggested timing

Table 1: A 75 minute breakdown.

Block	Time
Setup check, stragglers catch up	5 minutes
Interrupts, live	15 minutes
The bus hierarchy, live	20 minutes
Programmed I/O versus DMA, live	25 minutes
Wrap up and connection to Thursday	10 minutes

5. Wrap up

Everything today used commands that read from the system, nothing was changed or configured. The same categories, interrupts, bus topology, and the DMA offload we watched in `vmstat`, reappear Thursday on the Raspberry Pi units, where you will see interrupts tied to GPIO pins and a USB bus with far fewer devices on it than a laptop.

6. Reference, concept to command

Table 2: What each command reveals, and where it appears in the text.

Concept	Command	Textbook location
Interrupt cycle	<code>watch -n 1 cat /proc/interrupts</code>	Ch 3, Fig 3.9 to 3.14
Bus interconnection, PCIe	<code>lspci -tv</code>	Ch 3, interconnection and PCIe
USB external interface	<code>lsusb -tv</code>	Ch 8, external interfaces
DMA versus programmed I/O	<code>vmstat 1</code> during a large copy	Ch 8, Table 8.1, Fig 8.12
SATA DMA capability	<code>sudo hdparm -I /dev/sdX</code>	Ch 8, DMA

7. Appendix, install commands

If you did not complete the setup checklist before class, install the same five packages now.

Table 3: Install commands by distribution.

Distribution	Command
Arch Linux	<code>sudo pacman -S pciutils usbutils ethtool iw hdparm</code>
Gentoo	<code>sudo emerge -ask sys-apps/pciutils sys-apps/usbutils sys-apps/ethtool net-wireless/iw sys-apps/hdparm</code>
Void Linux	<code>sudo xbps-install -S pciutils usbutils ethtool iw hdparm</code>